UNIT-V

Syllabus:

Reinforcement learning - overview - Getting lost Examples Markov chain Monte Carlo Methods - sampling - Proposal distribution - Markov chain Monte Carlo - Graphical Models, Bayesian Networks - Markov Random Fields - Hidden Markov Models - Tracking Methods.

Reinforcement learning Overview:- [Sign lang]

It is a feedback-based ML approach here an agent learns to which actions to perform by looking at the Environment and the results of actions.

It will learn depending on the changes that are occuring in the environment.

Supervised - Human will Explicity teach the machine

Unsupervised - Computer itself will learn

Reinforcement - It will learn - depend on the changes which are happen in Env.

Goal - To achieve best result

eg: Chessboard [whatever moves we are doing is not known the game depends on the moving - the game will be change]

goal - to win the game

ex: Agent

(2 ways - fire and water)

Fire - wrong choice (-50 points)

water - Correct choice (+50 points)

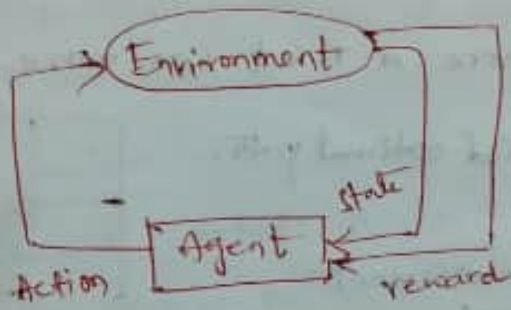
whenever choice is made (wrong or right) feedback is sent to agent.

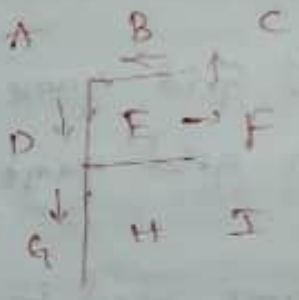
Feedback - based on this right choice is taken.

(or)

→ for each good point/action, the agent gets positive feedback, and for each bad action, the agent gets negative feedback or penalty.

→ In reinforcement learning, the agent learns automatically using feedback without any labeled data, unlike supervised





- It is used to solve specific type of problem where decision making is sequential, and the goal is long-term, such as game-playing, robotics etc.

Types :-

Positive Reinforcement learning :-

- It is a recurrence (same) of behavior due to positive rewards
- It increase strength and the frequency.
- It encourages to execute similar actions that yield maximum reward.

Negative Reinforcement learning:-

- Negative rewards are used as a deterrent to weaken the behavior and to avoid it.
- Decrease strength and frequency of a behavior.

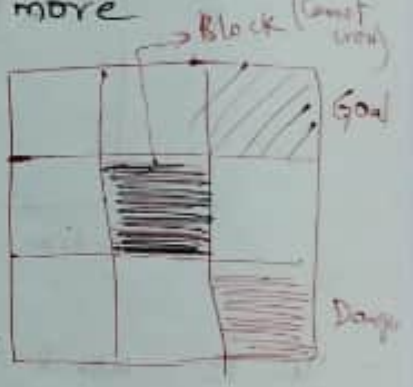
Example ① maze game

There may be a danger spot that may lead to loss

- Reinforcement learning is an example of semi-supervised learning technique and is used to model sequential decision-making process.

② Grid game - where a robot can move

③ Grid game - find optimal path.



Terms used in Reinforcement

① **Agent()** - An entity that can perceive / explore the environment and act upon it.

Environment():- A situation in which an agent is present or surrounded by. Random in nature.

Action() - Actions are the moves taken by an agent within the environment.

State() - State is a situation returned by the env after each action taken by the agent.

Reward() - A feedback returned to the agent from the environment to evaluate the action of the agent

policy() - policy is a strategy applied by the agent for the next action based on the current state.

To solve the problems of Reinforcement learning we will use the Bellman equation.

Getting lost Examples:-

In reinforcement learning, "getting lost" can refer to situations where the agent fails to find the optimal path or policy, often due to poor exploration or insufficient training.

① Cartpole problem:

Desc: The agent controls a cart that moves left or right to keep a pole balanced upright.

Getting lost: If the agent doesn't explore enough or gets stuck in a suboptimal policy it might fail to keep the pole balanced, resulting in the pole falling over.

② Mountain Car problem:

Desc: The agent must drive a car up a steep hill, but the car's engine is not strong enough to climb the hill directly.

Gettinglost - The agent might repeatedly try to go up the hill without enough momentum, failing to find the optimal strategy of going back and forth to build up enough speed.

③ Robot Navigation

Desc: The agent is a robot that needs to navigate from one point to another in an env with obstacles

Getting lost - The agent might stuck in a loop or repeatedly hit obstacles, failing to find a path to the goal.

④ Game playing

Desc: The agent plays a game like Chess or Go

Getting Lost - The agent might make poor moves due to insufficient exploration or lack of understanding of the dynamic dynamics, leading to lose the game.

⑤ Recommended System

Desc: The agent recommended products or content to users based on their preferences.

Getting lost - The agent might recommended irrelevant or repetative items, failing to improve user satisfaction

These examples, illustrate how learning agents can encounter challanges and 'get lost' during training. Proper Exploration, reward shaping, and sufficient training are crucial to help agents learn optimal policies and avoid getting stuck in suboptimal behaviors

Sampling - It is a way to approximately estimate certain characteristics of the whole population by taking a subset of the population into the study.

Markov Chain → Monte Carlo Sampling :- [Pizza, hamburger, hotdog]

MCMC sampling is a class of algorithms used to draw samples from a probability distribution. It's particularly useful when dealing with complex, high-dimensional distributions that are difficult to sample from directly.

MCMC - Sometimes estimating inference in the high-dimension can become computationally infeasible, in such case we resort to approximate it by MCMC

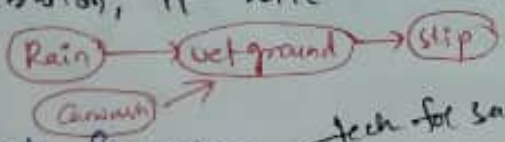
Key Concepts:-

① Markov chains [Assumption]

It is a sequence of random variables where the probability of each variable depends only on the previous one. In MCMC the chain is constructed so that its equilibrium distribution matches the target distribution.

↓ Probability of jumping from one state to next depends only on the current state not on the set of previous that lead to current

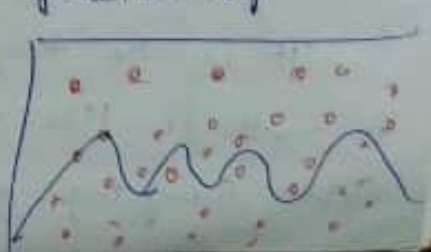
known as stationary distribution, that it is a probability distribution that remains unchanged as the system evolves over time. This means that the sys state in this distribution, it will remain in this after any number of steps.



② Monte Carlo Sampling — tech for sampling from a prob dist and using those samples to approximate desired quantity.

This involves using randomness to solve problems that might be deterministic in principles, In MCMC, it's used to sample from the target distribution by generating random samples [Quantity of Interest Estimation]

Eg:- Asked to Calculate the area under the curve. Randomly generate red dots in the rectangle and Cal the ratio of dots falling under the curve.



③ Convergence:

Overtime, the samples generated by the Markov Chain will converge to the target distribution, allowing for accurate estimation of its properties.

Algorithms

- Metropolis-Hastings Alg: $q(x'|x)$ is used to generate a new Candidate state

One of the most widely used MCMC alg. It involves proposing a new sample and accepting or rejecting it based on a certain criterion.

- Gibbs Sampling-

A special case of the Metropolis-Hastings alg where each variable is sampled sequentially from its conditional distribution.

Hamiltonian Monte Carlo

An advanced alg that uses gradient info to propose new samples, making it more efficient for certain types of distributions.

Proposal Distribution:

It is an essential component that suggests new candidate states for the Markov chain to move to during the sampling process.

It is the key element of the algorithm, particularly in methods like the Metropolis-Hastings and Gibbs Sampling algorithms.

The proposal distribution defines how new candidate points (states) are generated during the sampling process, based on the current state of the Markov chain.

Purpose:

The role of the proposal distribution is to suggest new states (or points) in the parameter space, from which the algorithm will either accept or reject based on certain criteria.

Its goal is to explore the target distribution effectively so that the chain will eventually converge to the desired distribution.

Types: Independent proposal - x' is drawn independently from a fixed distribution, not necessarily dependent on the current state x.

① Symmetric proposal distribution:-

The proposal distribution is symmetric if the probability of proposing a move from state x to y is the same as from state y to x'. i.e.

$$q(x \to y) = q(y \to x)$$

② Asymmetric proposal distribution:-

If the proposal distribution is asymmetric, then

$$q(x \to y) \neq q(y \to x)$$

and the acceptance probability needs to be adjusted accordingly.

③ Gaussian / Normal proposal -

In many cases, a Gaussian distribution centered account the current state is used as a proposal distribution. For example, you may propose a new state y from a Gaussian distribution.

$$y = x + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

where x is the current state

ϵ - is a random noise term.

④ Random walk proposal :-

A random walk can be used where the proposal distribution is a small step away from the current state. In this case, the proposed is typically generated as

$$y = x + \Delta \rightarrow x' = x + \epsilon$$

where Δ, is drawn from some distribution. (small random perturbation)

⑤ Multivariate proposal :-

In problems with multiple parameters, this proposal distribution might be a multivariate distribution, such as a multivariate Gaussian, which proposes changes to multiple parameters simultaneously.

Role :-

① Generating Candidates -

The proposal distribution, denoted as $q(x'|x)$, suggests a new candidate (x') and given the current state (x). It's responsibility for exploring the state space and

and ensuring that the Markov chain has the potential to visit all relevant regions of the target distribution.

(2) Acceptance Criterion:-

Once a candidate state 'x' is proposed, it is either accepted or rejected based on an acceptance criterion. The acceptance probability often depends on the target distribution and the proposal distribution, ensuring that the chain will converge to the target distribution.

Eg: Metropolis-Hastings Algorithm - $q(x'|x)$

It is used to generate a new candidate state. The candidate state is then accepted with a probability that ensures the Markov chain converges to the target distribution $\pi(x)$.

Acceptance probability:-

$$\alpha(x, x') = \min\left(1, \frac{\pi(x')\, q(x|x')}{\pi(x)\, q(x'|x)}\right)$$

(1) Propose a candidate

Generate a candidate state 'x' from the $q(x'|x)$.

(2) Compute acceptance probability

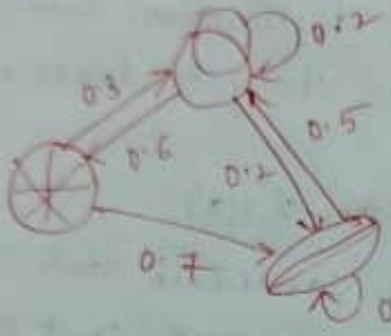
Calculate the Acceptance probability $\alpha(x, x')$.

(3) Accept or Reject:

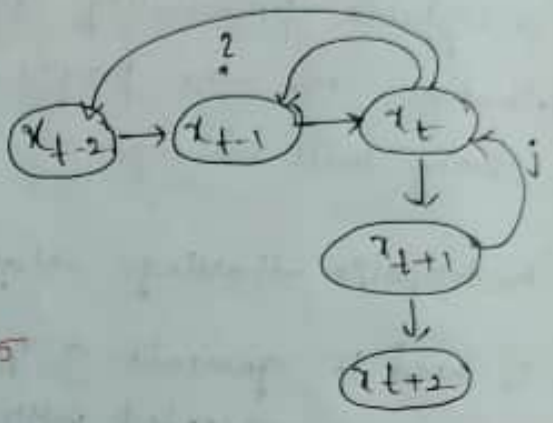
- Accept the candidate state 'x' with prob $\alpha(x, x')$
- If accepted, move to x'; Otherwise, stay at x.

Example of MCMC:- (Hamburger, pizza, hotdog)

On each day they will serve only one of these three items. And it depends on what they have served yesterday. And another way to predict what they will serve tomorrow if you know what there is serve today.



State convention



Property

① The future state depends only on the current state not the steps before. The probability

$$P(X_{n+1} = x \mid X_1 = x_1, X_2 = x_2 \cdots, X_n = x_n)$$

X_{n+1} step will be x depends only on the nth step not the complete sequence of steps that came before n. At first glance this might not seem so impressive

$$P(X_{n+1} = x \mid X_n = x_n).$$

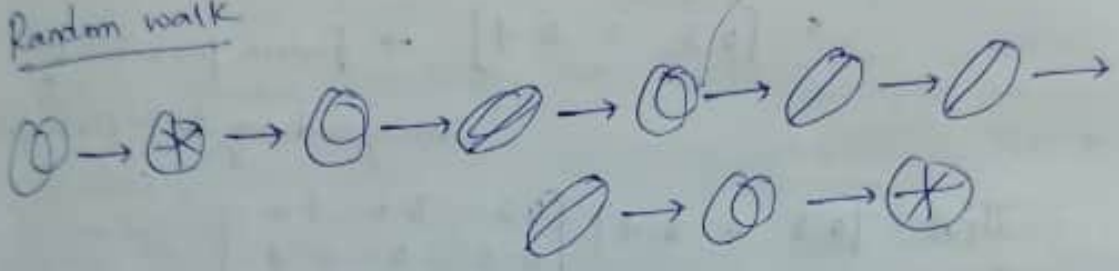
→ Suppose, the restaurant served pizza on first hamburger on second and again pizza on third, then what is the probability of serving hotdog on fourth day we will look out on the third day and it turns out to be 0.7.

$P(X_4 = \text{hot dog} \mid X_3 = \text{pizza}) = 0.7$

-this is the heart of the Markov chains and known as markov of property

② The sum of the weights of the outgoing arrows from any state is equal to 1. This has to be true. because they represent probabilities.

Random walk



No we want to find the probability corresponding to each of the food items

$P(\text{pizza})$ $P(\text{hot dog})$ $P(\text{ice cream})$ } This probability known as stationary distribution

$\frac{4}{10} = 0.35191$ $\frac{2}{10} = 0.21245$ $\frac{4}{10} = 0.43564$

In long walk the probabilities converge to fixed values or they continue to change forever.

Directed graph: pizza, hot dog, ice cream

$$A = \begin{matrix} \text{pizza} \\ \text{hot dog} \\ \text{ice cream} \end{matrix} \begin{bmatrix} 0.2 & 0.6 & 0.2 \\ 0.3 & 0 & 0.7 \\ 0.5 & 0 & 0.5 \end{bmatrix}$$

Our goal is to find probabilities of each state.

Let's Consider one row called as π_0 we are on a Pizza day.

$$\pi_0 = [0 \quad 1 \quad 0]$$

when we multiply this row vector with our transition matrix

$$\pi_0 A = [0 \quad 1 \quad 0] \begin{bmatrix} 0.2 & 0.6 & 0.2 \\ 0.3 & 0 & 0.7 \\ 0.5 & 0 & 0.5 \end{bmatrix}$$

$$= [0.3 \quad 0 \quad 0.7] \rightarrow \text{Future probability corresponding to the pizza state}$$

$$\pi_1 A = [0.3 \quad 0 \quad 0.7] \begin{bmatrix} 0.2 & 0.6 & 0.2 \\ 0.3 & 0 & 0.7 \\ 0.5 & 0 & 0.5 \end{bmatrix}$$

$$= [0.41 \quad 0.18 \quad 0.41]$$

$$\pi_2 A = [0.41 \quad 0.18 \quad 0.41] \begin{bmatrix} 0.2 & 0.6 & 0.2 \\ 0.3 & 0 & 0.7 \\ 0.5 & 0 & 0.5 \end{bmatrix}$$

$$= [0.34 \quad 0.25 \quad 0.41]$$

Now if there exist a stationary state after a certain point the output row vector should be exactly same as the i/p vector.

$$\boxed{\pi A = \pi}$$

$$0.2x + 0.3y + 0.5z = x$$

$$0.6x = y$$

$$0.2x + 0.7y + 0.5z = z$$

from $\pi[1] + \pi[2] + \pi[3] = 1$

$x + y + z = 1$

$\pi = [0.35211 \quad 0.21127 \quad 0.43662]$ - output

From $\pi A = \pi$

here pi equals to pi

If we have ever taken the codes of Algebra then we should find this equation somewhat similar to a very famous equation. i.e. Eigen vector equation

Consider, lambda = 1, there just reverse the order of the multiplication & got equilibrium state Equation. Interpret as

we can imagine that pi is a left eigenvector of matrix a with eigen value equal to 1.

$$A\vec{v} = \lambda\vec{v}$$

Then the Eigen vector must satisfy with one cond that is the elements of pi must add up to 1 because it denotes probability distribution.

After finding these two Eq^s we get the stationary state as this

$$\pi = \left[\frac{25}{71} \quad \frac{15}{71} \quad \frac{31}{71}\right] = [0.35 \quad 0.21 \quad 0.436]$$

From this the restaurant Serves hamburger about 35% of days, pizza 21% of days and hotdog 43% of days.

MCMC Graphical Models:-

It is the way of representing the conditional dependencies between random variables.

Graphical Models

These are visual representations of the conditional dependencies between variables. They can be represented as Bayesian n/w's (directed acyclic Graphs) or Markov random fields (undirected graphs)

Bayesian Network:-

Bayes theorem

It is a fundamental concept in ML; that plays a major role in various ML algorithms, especially in the fields of Bayesian statistics and probabilistic modelling.

It provides a way to update probabilities based on new evidence or information.

The theorem can be mathematically expressed as.

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

A Bayesian Network is a probabilistic graphical model which represents a set of variables and their conditional dependencies using a directed Acyclic Graph.

It is also called as Bayes network, belief n/w, decision n/w or Bayesian Model.

Here n/w are built from a probability distribution

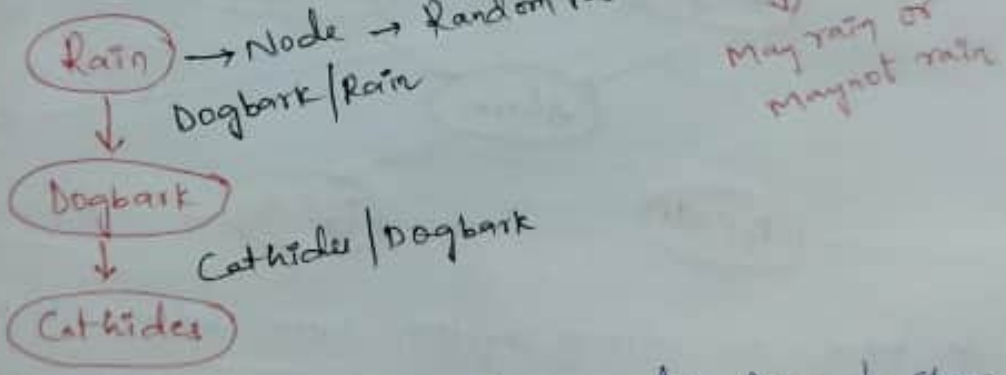
Applications:

- Prediction
- Anomaly detection
- Diagnostics
- Reasoning
- Time series prediction
- Decision making under uncertainity.

Before Going to learn about Bayesian n/w, we need to know the concepts

1) Directed Acyclic Graph (DAG)
2) Conditional probability Table (CPT)

DAG:



It is the graph where we have directions to show and it not contains any cycles in the Graph

CPT: Contains 48 events

	R	~R
B	9/48	18/48
~B	3/48	18/48

(B=T, & R=T) = 9/48 = 0.19

(B=T & R=F) = 18/48 = 0.375

(B=F & R=T) = 3/48 = 0.06

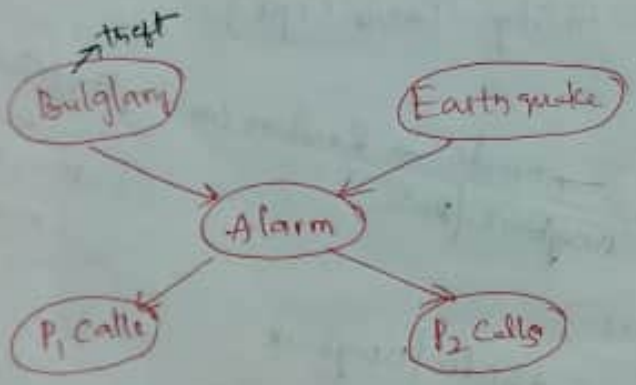
(B=F & R=F) = 14/48 = 0.375

Here why we are not Considering Cat hides, whenever the are calculating the probability, we should Calc the probability with respect to the parent node not with the respect to the child node.

Bayesian Belief Networks:

- Bayesian Belief n/w is a probabilistic graphical model (PGM) that represents conditional dependencies between random variables through DAG.
- It is also suitable for representing probabilistic relation b/w multiple events (more than 2 events).

Eg:



We have an alarm system here where if an Earthquake or in an bulglary it will send a notification to the person as P_1 Calls or P_2 Calls.

Given probabilities are,

$P(B=T) = 0.001$

$P(B=F) = 0.999$

$P(E=T) = 0.002$

$P(E=F) = 0.998$

Probability of Alarm

Bulgary (B)	Earthquake (E)	P(A) = T	P(A) = F
T	T	0.95	0.05
T	F	0.99	0.06
F	T	0.29	0.71
F	F	0.001	0.999

Probability of P_1

Alarm (A)	P(P_1 = T)	P(P_1 = F)
T	0.90	0.10
F	0.05	0.95

Probability of P_2

Alarm (A)	P(P_2 = T)	P(P_2 = F)
T	0.70	0.30
F	0.01	0.99

Q) find the probability of P_1 is T, P_2 is T, A is T, B is F, and E is F

i.e. $P(P_1, P_2, A, \sim B, \sim E)$.

$= P(P_1|A)\, P(P_2|A)\, P(A|\sim B, \sim E)\, P(\sim B)\, P(\sim E)$

$= 0.90 * 0.70 \times 0.001 \times 0.999 \times 0.998$

$= 0.000628$

When there is no earthquake and no burglary the probability of ringing the alarm is very very less, then the probability of ringing both the clan are also very less.

Markov Random fields: [used in undirected graph]

MRF's are a type of undirected graphical model used in Machine learning to represent the dependencies between random variables. They are particularly useful in areas like Image processing, Computer vision, and Spatial data analysis.

The undirected graphical models edge represents the potential b/w two variables, syntactically, factorization distribution probabilities between variables.

In each individual variable connected with the edge present a certain clique in the graph, means Probability distribution of the variables in the graph can factorize an individual clique potential function.

Just as we had CPD's for Bayesian Networks we have tables to incorporate relations between nodes in Markov networks.

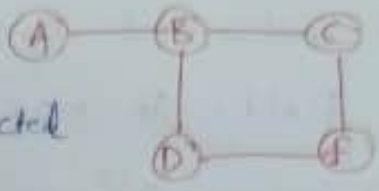
However, there are two crucial differences b/w these tables and CPD's [Conditional probability Distribution]

Clique- Vertex induced subgraph which is a complete graph. $V = \{1,2,3,4,5,6,7\}$

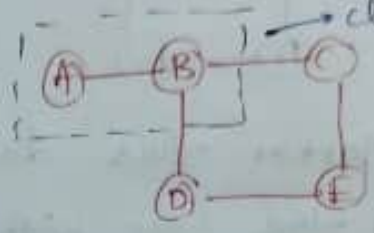


$V' = \{1,2,3,4\}$

Clique in graph theory, it is a subset of vertices of an undirected graph.



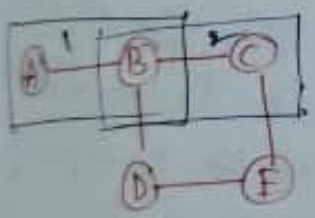
$P(A,B,C,D,E) \propto \phi(A,B)\ \phi(B,C)\ \phi(B,D), \phi(C,E)\ \phi(D,E)$

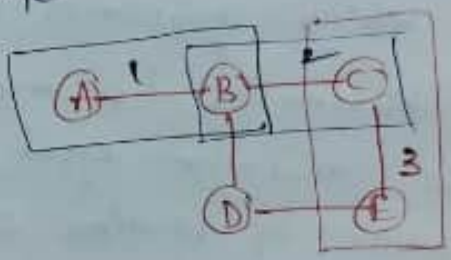


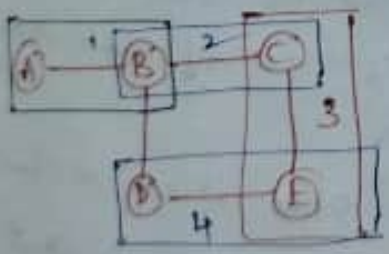
$$P(X) = \frac{1}{Z} \prod_{\text{clique}} \phi_c(x_c)$$

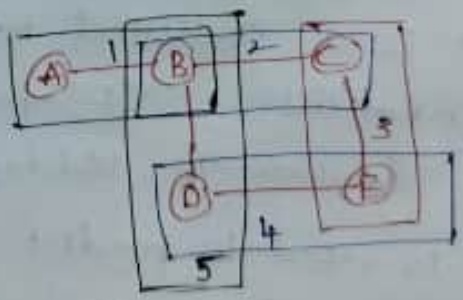
↓
potential functions

Such that, it induces sub graph is complete in every vertices in a clique is adjacent. So, clique in this graph adjust adjacently one by one.





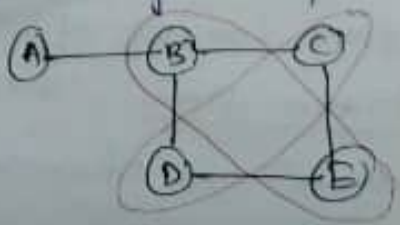




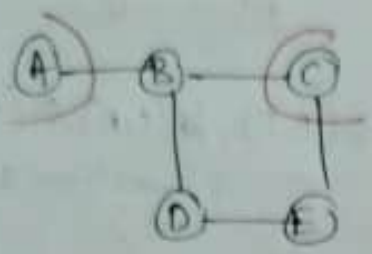
If it is different if we join D, C and B, E clique over here then it is also change its probability

$P(A,B,C,D,E)$

$\propto \phi(A,B)\ \phi(B,C,D), \phi(C,D,E)$



Some undirected graphical model has Markov Random field. In MRF certain paths b/w A and C



A → B → C

A → B → D → E → C

Note: Markov r/w's do not need to be acyclic, as was the case with Bayesian Networks.

Hidden Markov Models (HMM)

A Hidden Markov Model (HMM) is a probabilistic model that consists of a sequence of hidden states, each of which generates an observation.

The hidden states are usually not directly observable and the goal of HMM is to estimate the sequence of hidden states based on a sequence of observations.

It is a statistical model that is used to describe the probabilistic r/s b/w a sequence of observations and a sequence of hidden states.

It is used to predict future observations or classify sequences, based on the underlying hidden process that generates the data. [If we say the o/p, we can say the i/p in which the i/p is generated.

HMM Consists of two types of variables

① Hidden states: These are underlying variables that generate the observed data, but they are not directly observable.

② Observations: These are the variables that are measured and observed.

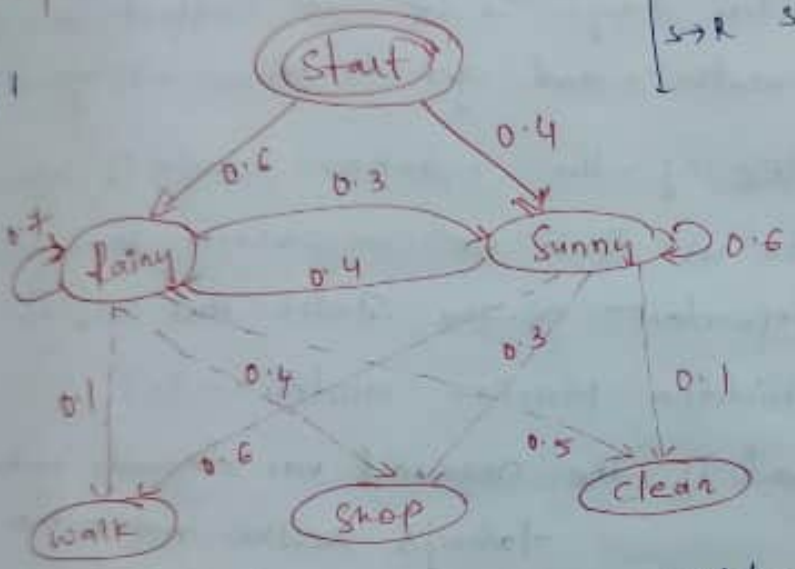
The relationship b/w the hidden states and the observations is modeled using a probability distribution.

HMM are used to generate other models based on the inputs.

Transition

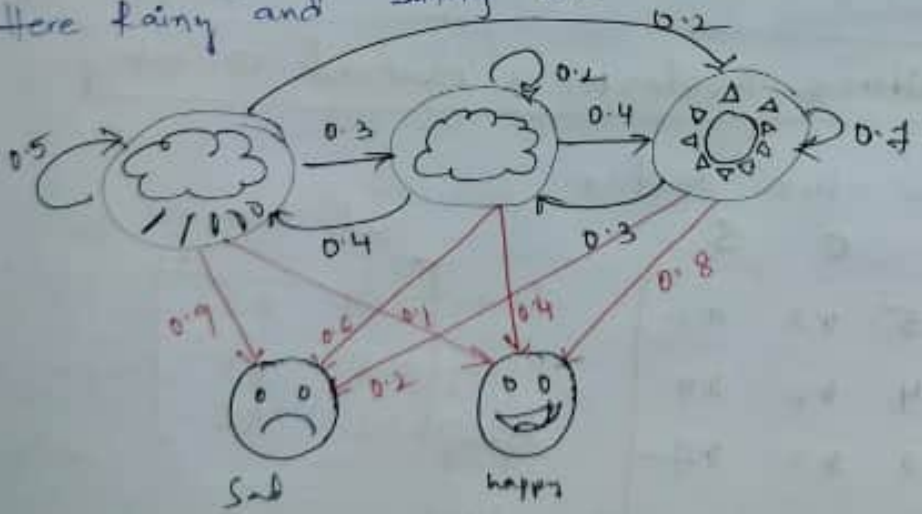
$$A = \begin{bmatrix} R \to R & R \to S \\ S \to R & S \to S \end{bmatrix} = \begin{bmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{bmatrix}$$

Eg 1



-Here Rainy and Sunny are hidden states.

Eg 2



* From this, a man Jack is living in a country where the weather changes like this Rainy, Cloudy and Sunny. On any given day only one of them will occur.

The next day will depend on the weather today

→ Assume, on any given day Jack has the following mood happy & Sad. Moreover this mood is dependent on the weather of the particular day.

--Here, the problem is we are not living in that Country, So we can't possible know what's the weather on the particular day. So we can contact Jack to know the weather, and get to know his mood.

→ So, the states of the markov chain is unknown or hidden to us. But we can observe some variable that are dependent on the states. And this is known as Hidden Markov Model.

Jack mood is the observed var depends on the today's weather not on the previous day.

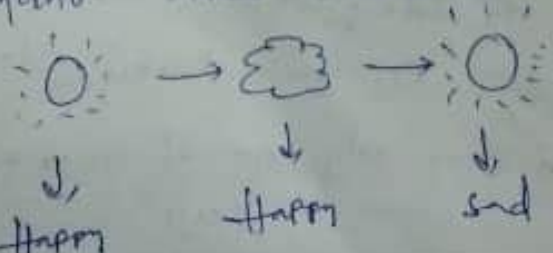
HMM = Hidden MC + Observed variable

Consider the matrix.

	R	C	S
Rainy	0.5	0.3	0.2
Cloudy	0.4	0.2	0.4
Sunny	0.0	0.3	0.7

	Sad	Happy
Rainy	0.9	0.1
Cloudy	0.6	0.4
Sunny	0.2	0.8

Scenario- Three Consecutive days



Assume this scenario happens.

b) what is the probability of this scenario occurring. precisely, what is the joint probability of the observed mood sequence and the weather sequence.

$P(Y = 😊 😊 😢, X = ☀ ☁ ☀)$

By using Markov property, we can compute this as the product of six terms.

$P(X_1 = \text{Sunny})$ - ? 0.00391

$P(Y_1 = \text{happy} \mid X_1 = \text{Sunny})$ - The prob of happy mood given a sunny day) 0.8

$P(X_2 = \text{cloudy} \mid X_1 = \text{Sunny})$ 0.3 $P(Y_2 = \text{happy} \mid X_2 = \text{cloudy})$ 0.4

$P(X_3 = \text{Sunny} \mid X_2 = \text{cloudy})$ 0.4 $P(Y_3 = \text{Sad} \mid X_3 = \text{Sunny})$ 0.2

- But how got this six terms
- To find $P(X_1 = \text{Sunny})$ - we need Stationary distribution of Markov chain we need (or) use normalised left eigen vectors or repeated matrix multiplication to compute the stationary distribution.

Key Components

① **States:** $S = \{S_1, S_2 \cdots, S_N\}$

The system can be in one of these N states at any given time.

② **Observations:** $O = \{O_1, O_2 \cdots, O_N\}$

These are the observed events or data points over time.

③ Transition probabilities:

- $A = \{a_{ij}\}$
- The probability of transitioning from state S_i to state S_j

④ Emission probabilities

- $B = \{b_j(O)\}$

The probability of observing 'O' given the system is in state S_j.

⑤ Initial state probabilities:

- $\pi = \{\pi_i\}$
- The probability of the system starting in state S_i.

Eg1:

Emission probabilities (B)

$$B = \begin{bmatrix} P(w|R) & P(H|R) & P(C|R) \\ P(w|S) & P(H|S) & P(C|S) \end{bmatrix}$$

$$= \begin{bmatrix} 0.1 & 0.4 & 0.5 \\ 0.6 & 0.3 & 0.1 \end{bmatrix}$$

Initial state probabilities

$$\pi = \begin{bmatrix} P(R) \\ P(S) \end{bmatrix} = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$$

Observations -

$$O = [w, H, C]$$

Algorithm:

1) Define the state space and observation space.
2) Define the initial state distribution
3) Define the state transition probabilities
4) Define the observation likelihoods
5) Train the model
6) Decode the most likely sequence of hidden states
7) Evaluate the model.

Implementation

Predicting the weather

1) Import the required libraries
2) Define model parameters
3) create an instance of the HMM model and set the model parameters
4) Define the observation sequence
5) predict the most likely sequence of hidden states.
6) Decode the observation sequence
7) Plot the results

Applications:

① Speech Recognition - Recognize spoken words
② Bioinformatics - Analyzing seq of DNA or proteins to identify genes to predict structures
③ finance - Modeling market states to predict stock prices or economic indicators.